MARK HANEY

STAFF SOFTWARE ENGINEER | PYTHON + REACT FULL-STACK ENGINEER | CLOUD, MICROSERVICES, CI/CD

Dardanelle, AR
https://www.linkedin.com/in/markhaney-33b675ba
+1 (430) 279-2231 | markhaney.dev@outlook.com

PROFESSIONAL SUMMARY

Staff Software Engineer with 10+ years of experience building **Python 3.8–3.12** and **React 16–18** applications across **SaaS, PropTech, GovTech, nonprofit CRM, and cloud service industries**, specializing in scalable APIs, secure microservices, frontend modernization, CI/CD automation, and production reliability while improving system performance, reducing defects, mentoring engineers, and delivering maintainable platforms with **FastAPI 0.95–0.115**, **Django 2.2–4.2**, **TypeScript 4.x–5.x**, **AWS**, **Docker**, and **Kubernetes**.

TECH SKILLS

- **Backend: Python 3.8–3.12**, **FastAPI 0.95–0.115**, **Django 2.2–4.2**, Flask 1.x–2.x, REST APIs, GraphQL, Celery, SQLAlchemy, PyTest
- **Frontend: React 16–18**, **TypeScript 4.x–5.x**, JavaScript ES6+, Redux Toolkit, React Query, Next.js 12–14, Material UI, Tailwind CSS
- **Databases:** PostgreSQL, MongoDB, MySQL, Redis, Elasticsearch
- **Cloud & DevOps: AWS**, Docker, Kubernetes, GitHub Actions, GitLab CI, Terraform, Nginx, Linux, CI/CD, Blue-Green Deployment
- **Engineering Practices:** Microservices, System Design, Agile, Unit Testing, Integration Testing, Code Review, Observability, Performance Tuning

PROFESSIONAL EXPERIENCE

Staff Software Engineer

Relatio

Jun 2022 – Apr 2026
Las Vegas, NV

- **Python 3.10–3.12** backend services were redesigned with **FastAPI 0.100+**, async workers, and PostgreSQL indexing, improving high-volume request throughput by **23%** while keeping APIs easier to maintain.
- **React 18** and **TypeScript 5.x** frontend modules were rebuilt with reusable hooks, typed API clients, and cleaner state boundaries, reducing duplicated UI logic by **19%** across core customer workflows.
- Legacy Django endpoints were gradually migrated into **FastAPI microservices** using strangler-pattern routing, contract tests, and feature flags, reducing release risk during modernization.
- Production latency issues caused by slow aggregation queries were traced through **OpenTelemetry**, optimized with query plans and Redis caching, and reduced page-load delays by **27%**.
- Authentication flows were strengthened with JWT rotation, role-based authorization, and backend permission guards, improving security consistency across internal and customer-facing modules.
- CI/CD pipelines in **GitHub Actions** were rebuilt with test gates, Docker image scanning, and staged deployments, reducing failed deployments by **21%**.
- React form validation bugs caused by inconsistent backend schemas were solved by introducing shared **OpenAPI-generated TypeScript clients** and stricter payload validation.

- Containerized services were standardized with **Docker** and Kubernetes deployment manifests, allowing engineering teams to reproduce production-like issues locally with less setup friction.
- Background task failures in Celery queues were resolved by adding retry policies, dead-letter handling, and structured logs, improving operational visibility during incident response.
- Cross-team architecture reviews were led for API versioning, caching strategy, and frontend state management, helping engineers make consistent decisions without slowing delivery.
- Performance improvements were measured through Datadog dashboards, synthetic checks, and release comparison reports, giving product owners clear visibility into stability gains.
- Junior and mid-level engineers were mentored through code reviews, pairing sessions, and design discussions focused on **clean architecture**, testing habits, and long-term maintainability.

Senior Software Engineer

AppFolio

Feb 2018 – May 2022
Goleta, CA

- Customer-facing property management features were developed with **Python 3.8–3.10**, **Django 3.x–4.x**, and **React 16–17**, improving workflow completion speed by **18%**.
- Large React screens were refactored from class components into functional components with hooks, reducing frontend defect reports by **16%** across repeated leasing workflows.
- Django REST APIs were optimized with serializer cleanup, pagination, select-related queries, and cache-aware endpoints, reducing database load during peak usage.
- Payment and document workflows were stabilized by adding idempotency keys, backend validation, and integration tests around edge cases that previously caused duplicate submissions.
- A legacy JavaScript dashboard was migrated into **React 17** with TypeScript-safe components, allowing new features to ship without breaking older customer screens.
- Production bugs caused by inconsistent timezone handling were fixed by normalizing UTC storage, improving date rendering accuracy across billing and reporting modules.
- AWS-based deployment workflows were improved with containerized builds, automated smoke tests, and safer rollback steps, reducing release recovery time by **22%**.
- Shared UI components were introduced for tables, filters, modals, and form controls, improving frontend consistency and reducing repetitive implementation work.
- Feature flags were used to release large backend and frontend changes incrementally, allowing product teams to validate behavior before full customer rollout.
- Agile delivery was supported through technical planning, estimation, peer reviews, and clear communication between engineering, QA, product, and support teams.

Software Engineer

Blackbaud

Jul 2016 – Jan 2018
Charleston, SC

- Nonprofit CRM and fundraising modules were enhanced using **Python 3.6–3.8**, Django, JavaScript, and early React components, improving donor workflow reliability by **14%**.
- REST API defects around duplicate donor records were solved by adding stronger uniqueness checks, transaction handling, and regression tests for import workflows.
- Reporting screens were improved with reusable frontend components, cleaner filtering logic, and optimized backend queries, reducing report generation delays by **17%**.

- Legacy server-rendered pages were incrementally converted into React-based widgets where interactive behavior mattered most, avoiding a risky full rewrite.
- Data migration scripts were created for older donor and campaign records, with validation reports used to detect missing fields before production rollout.
- Unit and integration test coverage was expanded with PyTest and Django test utilities, helping catch payment, import, and permission issues before release.
- Role-based access issues were debugged by tracing permission inheritance across organizations, campaigns, and user groups, then simplifying authorization checks.
- Release support included monitoring logs, reviewing failed jobs, and coordinating fixes with QA when customer-impacting issues appeared after deployment.
- Code review feedback focused on readable Python services, maintainable frontend structure, and reducing hidden coupling between reporting and donor modules.

Software Developer

Granicus

May 2014 – Jun 2016
Dallas, TX

- Government-facing workflow applications were built with **Python 2.7–3.5**, Flask, Django, JavaScript, and jQuery, improving public-service request handling by **13%**.
- Internal admin screens were upgraded with cleaner JavaScript modules, reusable form validation, and safer backend endpoints for government staff operations.
- Legacy Python scripts used for batch data processing were refactored into clearer service modules, reducing recurring job failures by **15%**.
- Slow API responses were investigated through log analysis, SQL query review, and endpoint profiling, then improved with indexing and response payload cleanup.
- Authentication and session issues were fixed by tightening cookie handling, improving server-side validation, and documenting login edge cases for QA.
- Early frontend modernization work introduced component-style JavaScript patterns before larger SPA adoption became standard across the platform.
- Deployment issues caused by environment mismatch were reduced through clearer configuration files, release checklists, and repeatable Linux-based setup steps.
- Production support work included diagnosing customer-reported bugs, preparing hotfixes, and communicating root causes in a practical way for nontechnical stakeholders.

Junior Software Developer

Vology

Jun 2013 – Apr 2014
Gainesville, FL

- Internal service tools were developed with **Python 2.7**, Flask, JavaScript, HTML, CSS, and MySQL, helping support teams process operational requests more efficiently.
- Repeated manual reporting tasks were automated with Python scripts and scheduled jobs, reducing weekly data preparation effort by **12%**.
- Frontend bugs in legacy JavaScript forms were fixed by improving validation, correcting event-handling issues, and testing behavior across common browsers.
- Database issues were resolved by cleaning inconsistent records, improving SQL queries, and adding safer backend checks before saving user-submitted data.

- Senior engineers were supported through debugging, unit test updates, documentation cleanup, and small feature delivery in an Agile development environment.

EDUCATION

Arkansas State University
Bachelor's degree, Computer Science (Sep 2009 – May 2013)